

Программа курса «Углублённый Python»

Курс рассчитан на студентов экономического факультета, для которых программирование не является профильной дисциплиной, но которые осознанно выбрали углублённый курс.

Предполагается, что слушатели уже писали программы хотя бы на одном языке. Знание именно Python не требуется: первая лекция даёт компактный обзор языка, после чего программа переходит к его модели данных, алгоритмам, структурам данных и практике разработки приложений.

Входной тест: на любом языке программирования решить следующую задачу. На входе строка из букв латинского алфавита, цифр и пробелов, нужно вывести самое длинное слово в ней. Пробелов может быть несколько подряд, длина строки ≤ 2000 символов.

Формат курса

- 14 лекций и 14 семинаров;
- 10 практических семинаров;
- 3 справочных семинара без обязательной практической части;
- 1 семинар, посвящённый защите групповых проектов;
- 13 домашних работ: после каждой лекции, кроме последней;
- групповой проект по одной из предложенных преподавателем тем;
- факультативный материал (опционально)

Материалы каждого занятия размещаются в отдельном каталоге `lesson??`:

```
lesson01/
├── README.md
├── lecture.ipynb
├── seminar.ipynb
└── tasks.md
```

Каждая домашняя работа содержит:

- 3 задачи уровня Easy;
- 2 задачи уровня Medium;
- иногда 1 необязательную задачу уровня Hard.

Названия уровней обозначают сложность, сопоставимую с задачами LeetCode. Для прикладных тем уровни обозначают сложность инженерной работы, а не алгоритмической головоломки.

Тематический план

№	Лекция	Семинар	Формат семинара	Домашняя работа
1	Обзор Python. Модель выполнения, скалярные типы, выражения, truthiness, <code>if/elif/else</code> , <code>for</code> , <code>while</code> , <code>break</code> , <code>continue</code> , функции, параметры, <code>return</code> , области видимости и базовые аннотации типов	Небольшие функции с ветвлениями и циклами; перевод знакомых алгоритмических конструкций на Python	Практический	3 Easy + 2 Medium
2	Последовательности и работа с файлами. Строки и списки, индексы, срезы и comprehensions; <code>match/case</code> ; <code>pathlib</code> , текстовые файлы, кодировки, контекстные менеджеры, CSV/JSON и явная обработка ошибок	Чтение, проверка, преобразование и сохранение небольшого набора данных	Практический	3 Easy + 2 Medium
3	Алгоритмы и сложность. Время и память, асимптотическая нотация, лучший и худший случаи, линейный и бинарный поиск	Реализация, тестирование и экспериментальное сравнение алгоритмов поиска	Практический	3 Easy + 2 Medium + Hard
4	Применение Python. Скрипты и автоматизация, анализ данных и машинное обучение, веб-приложения и API, боты, игры и симуляции, применение ИИ, границы применимости Python	Git, окружения, зависимости, модули и пакеты, <code>pyproject.toml</code> , <code>src-layout</code> , CLI, <code>ruff</code> , smoke-тесты и разрешение merge conflict	Практический	3 Easy + 2 Medium
5	Модель данных и контейнеры. Объекты, ссылки, равенство и идентичность, изменяемость, <code>tuple</code> , <code>dict</code> , <code>set</code> , хешируемость, поверхностное и вложенное копирование, точные вычисления через <code>Decimal</code>	Диагностика общих ссылок, выбор контейнера и построение независимых сценариев данных	Практический	3 Easy + 2 Medium
6	Алгоритмы сортировки. Стабильность, сортировка по ключу, insertion sort, merge	Реализация и сравнение сортировок	Практический	3 Easy + 2 Medium + Hard

№	Лекция	Семинар	Формат семинара	Домашняя работа
	sort, идея quicksort и встроенный Timsort			
7	Итераторы и генераторы. Протокол iterable/iterator, <code>iter</code> , <code>next</code> , <code>StopIteration</code> , генераторные функции и выражения, <code>yield</code> , <code>yield from</code> , ленивые вычисления, <code>collections</code> и <code>itertools</code>	Ленивый конвейер преобразования данных без загрузки всего набора в память	Практический	3 Easy + 2 Medium
8	Стек и очередь. LIFO, FIFO, <code>list</code> , <code>deque</code> , амортизированная сложность и типовые применения	Скобочные последовательности, очередь заявок, история отмены и скользящее окно	Практический	3 Easy + 2 Medium + Hard
9	Объектная модель и качество кода. Классы, <code>dataclass</code> , композиция, наследование, протоколы, аннотации типов и основы декораторов	Демонстрации <code>pytest</code> , параметризации, fixtures, mocking, логирования и конфигурации; разбор проблем групповых проектов	Справочный	3 Easy + 2 Medium
10	Хеш-таблицы. Хеш-функции, коллизии, коэффициент заполнения, требования к ключам, сложность <code>dict</code> и <code>set</code>	Частотный анализ, дедупликация, индексирование и кэширование	Практический	3 Easy + 2 Medium + Hard
11	Деревья и кучи. Бинарные деревья поиска, DFS/BFS, рекурсия, <code>heapq</code> и очередь с приоритетом	Обходы дерева, поиск, <code>top-k</code> и планировщик задач	Практический	3 Easy + 2 Medium + Hard
12	HTTP и асинхронное программирование. HTTP/JSON, CPU-bound и I/O-bound задачи, coroutine, task, event loop, <code>await</code> , timeout, cancellation и semaphore	Демонстрации HTTP-клиентов, конкурентного исполнения, тайм-аутов и ограничения параллелизма	Справочный	3 Easy + 2 Medium + Hard
13	Реляционные базы данных. Таблицы, ключи, ограничения, SQL, <code>JOIN</code> , агрегации, транзакции, индексы, SQLite и основы SQLAlchemy/SQLModel	Интерактивный SQL, подключение БД из Python и консультации по схемам данных проектов	Справочный	3 Easy + 2 Medium

№	Лекция	Семинар	Формат семинара	Домашняя работа
14	FastAPI, Docker и поставка приложения. REST API, Pydantic, dependency injection, тестирование API, Dockerfile, Compose, переменные окружения, healthcheck, CI, запуск и принципы деплоя	Защита групповых проектов	Защита	Нет

Содержание занятия 4

Четвёртая лекция показывает основные области применения Python и помогает студентам осознанно выбрать проект. Она не заменяет последующие специальные темы: машинное обучение, FastAPI, асинхронность и ИИ рассматриваются обзорно, через типовые задачи и архитектуру приложений.

На четвёртом семинаре студенты превращают код предыдущих занятий в устанавливаемый проект примерно следующей структуры:

```

project/
├── pyproject.toml
├── README.md
├── src/
│   └── project_name/
│       ├── __init__.py
│       ├── algorithms.py
│       ├── io.py
│       └── cli.py
└── tests/
    └── test_smoke.py

```

Минимальный результат семинара:

- предметная логика отделена от ввода-вывода и CLI;
- пакет устанавливается в чистое виртуальное окружение;
- CLI запускается через console entry point;
- **ruff** и smoke-тесты выполняются одной воспроизводимой последовательностью команд;
- работа выполнена в отдельных ветках, учебный merge conflict разрешён;
- команды установки, проверки и запуска записаны в README.

Самостоятельная работа

В курсе предусмотрено 65 основных домашних задач и 6 необязательных задач уровня Hard. Бонусные задачи запланированы после занятий 3, 6, 8, 10, 11 и 12.

Рекомендуемая модель освоения набора:

- базовый уровень: 3 Easy;
- хороший уровень: 3 Easy и 1 Medium;
- высокий уровень: 3 Easy и 2 Medium;
- Hard: необязательное усложнение.

На занятиях 4, 9, 13 и 14 часть инженерной работы может выполняться непосредственно в групповом проекте, чтобы проект и домашние задания не дублировали друг друга.

Групповой проект

Предметную область и конкретную задачу каждая группа выбирает из списка, подготовленного преподавателем. Общего для всех сквозного сюжета нет.

Технические требования к проекту

Обязательные требования:

- репозиторий с осмысленной историей Git;
- воспроизводимое окружение и инструкция запуска;
- предметное ядро, отделённое от внешних интерфейсов;
- аннотации типов и явная обработка ошибок;
- автоматические тесты;
- обоснованное применение изученных алгоритмов, структур данных или инженерных решений;
- README с описанием задачи, архитектуры и выбранного технологического стека.

Конкретные технологии выбираются группой в соответствии с задачей проекта и согласуются с преподавателем. В проекте могут использоваться, например:

- CLI, веб-интерфейс, REST API, бот или иной пользовательский интерфейс;
- файлы, SQLite, PostgreSQL, внешнее API или другое хранилище данных;
- FastAPI, Flask, Streamlit или другой подходящий фреймворк;
- синхронная или асинхронная модель выполнения;
- Docker и CI, если они упрощают воспроизводимый запуск и поставку приложения.

FastAPI, SQL, асинхронность и Docker не являются обязательными сами по себе. Оценивается не количество использованных технологий, а соответствие решений задаче, качество реализации и способность группы объяснить свой выбор.

Контрольные точки

1. После занятия 4: выбор проекта, постановка задачи и создание репозитория.
2. После занятия 7: предметное ядро и первоначальная архитектура.
3. После занятия 9: структура приложения, тесты, типизация и логирование.
4. После занятия 11: алгоритмическая часть и обоснование сложности.
5. После занятия 13: выбранные внешние интерфейсы и хранение данных, если они нужны проекту; обоснование технологических решений.
6. Семинар 14: финальная демонстрация и защита.

Материалы по FastAPI и Docker можно открыть группам заранее как справочник. Лекция 14 систематизирует способы поставки приложения, но не вводит обязательный для всех стек.

Справочные семинары

Семинары 9, 12 и 13 не содержат обязательной сдаваемой практической части. Их задача — дать рабочие примеры, разобрать типичные ошибки и помочь применить технологии в проектах. Домашние работы после соответствующих лекций сохраняются.

Факультативные материалы (список не окончательный)

- создание MCP-сервера над готовым предметным ядром проекта;
- NumPy, pandas и визуализация данных;
- PostgreSQL и многоконтейнерный запуск через Docker Compose;
- расширенные возможности ООП;
- дополнительные алгоритмы и структуры данных.